

Experiment No: 03

Experiment Name: Design of an LQR Controller for a Given System Using Trial & Error Method

Objectives:

1. To design PID controller, PD controller, and PI controller for given systems using the Root Locus method.
2. To evaluate and analyze the performance of these controllers based on their Root Locus plots.
3. To adjust and optimize the PID controller parameters to achieve the desired system performance.

Theory:

Linear Quadratic Regulator (LQR):

The Linear Quadratic Regulator (LQR) is an optimal control strategy designed for linear systems to minimize a quadratic cost function, typically representing a trade-off between tracking performance and control energy. It provides a systematic methodology for calculating a feedback gain matrix K that ensures the stability and optimality of the system's state regulation or tracking.

LQR Controller Block Diagram:

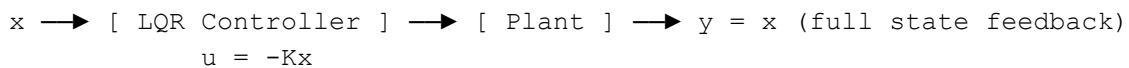


Fig 3.1: Schematic of an LQR Controller.

Proportional-Integral-Derivative (PID):

A Proportional-Integral-Derivative (PID) controller is a three-term feedback mechanism that generates a control signal based on the error between a desired setpoint and a measured process variable. The Proportional (P) term provides an action proportional to the current error. The Integral (I) term reduces steady-state error by accumulating past errors. The Derivative (D) term improves transient response by predicting future error based on its current rate of change.

PID Controller Block Diagram:

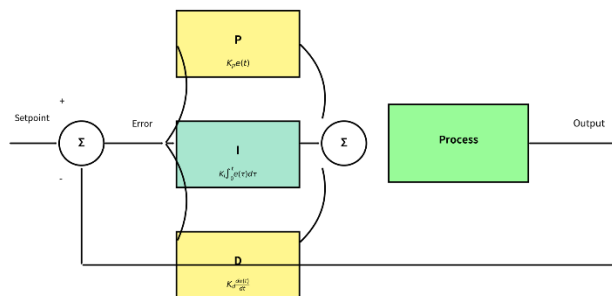


Fig 3.2: Block diagram for PID controller.

Root Locus:

The Root Locus method is a graphical technique used in control system design to show how the roots (poles) of the closed-loop characteristic equation move in the complex s-plane as a specific parameter, typically the loop gain k , varies from zero to infinity. It allows designers to evaluate system stability and transient performance visually by observing the trajectories of these poles.

Trial and Error Method:

In the context of LQR design, trial and error refers to the iterative process of manually adjusting the weighting matrices Q (state penalty) and R (control penalty) to meet specific performance criteria such as rise time, overshoot, or settling time. Because the relationship between the weights in Q and R and the resulting time-domain performance is often non-linear and complex, designers frequently start with Bryson's Rule or an identity matrix and then tune the diagonal elements until the system response is optimized.

Required Apparatus:

1. MATLAB 2024a.
2. Laptop.

System Model:

The system under study is a DC motor with armature resistance R , armature inductance L , back-EMF constant K , moment of inertia J , and viscous damping b . The armature circuit drives a rotor connected to a fixed magnetic field.

DC Motor Circuit (Armature Model):

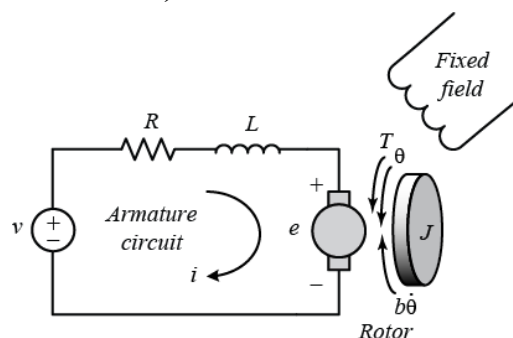


Fig 3.3: DC motor circuit.

Transfer Function:

Transfer Function of DC Motor: $\theta'(s)/V(s) = K / [(R + sL)(Js + b) + K^2]$

Transfer Function of PID Controller = $KD[s^2 + (Kp/KD)s + (Ki/KD)] / s$

Transfer Function of PI Controller = $(sKp + Ki) / s$

Transfer Function of PD Controller = $KD [s + Kp/KD]$

Code:

```

clc;
clear;
close all;

% System Parameters (DC Motor)
J = 0.01;
b = 0.1;
K = 0.01;
R = 1;
L = 0.5;
s = tf('s');

% Transfer Function
TF = K/((J*s + b)*(L*s + R) + K^2);

%% PID Controller
Kp_pid = 800;
Ki_pid = 500;
Kd_pid = 50;
PID = pid(Kp_pid, Ki_pid, Kd_pid);
CL_PID = feedback(PID*TF, 1);

%% PI Controller
Kp_pi = 400;
Ki_pi = 250;
PI = pid(Kp_pi, Ki_pi, 0);
CL_PI = feedback(PI*TF, 1);

%% PD Controller
Kp_pd = 1000;
Kd_pd = 20;
PD = pid(Kp_pd, 0, Kd_pd);
CL_PD = feedback(PD*TF, 1);

%% Step Responses
figure;
step(CL_PID, CL_PI, CL_PD);
legend('PID (to 1.0)', 'PI (to 1.0)', 'PD (settles < 1.0)');
title('Closed Loop Step Response Comparison');
grid on;

%% Performance Info & Delay Time Calculation
controllers = {CL_PID, CL_PI, CL_PD};
names = {'PID', 'PI', 'PD'};
for i = 1:3
    fprintf('\n--- %s ---\n', names{i});
    info = stepinfo(controllers{i});
    [y, t] = step(controllers{i});
    final_val = y(end);
    delay_index = find(y >= 0.5 * final_val, 1);
    delay_time = t(delay_index);
    disp(info);
    fprintf('DelayTime: %.4f seconds\n', delay_time);
end

```

Procedure: Obtaining Controller Parameters Using Root Locus Method

The Root Locus method provides a systematic approach to place closed-loop poles at desired locations in the s-plane by adjusting controller gains. The following steps outline how to design PID, PI, and PD controllers for the DC motor system.

Step 1: Define System Specifications

Before beginning the design, establish the desired performance specifications: maximum percent overshoot (%), settling time (s), rise time (s), and steady-state error. These translate directly into target pole locations on the s-plane.

Step 2: Plot the Open-Loop Root Locus

Enter the plant transfer function TF in MATLAB and plot the open-loop root locus using `rlocus(TF)`. Identify the locations of open-loop poles and zeros on the s-plane. The root locus branches begin at the poles and end at the zeros (or infinity). Observe whether the locus passes through the desired closed-loop pole region.

Step 3: Add Controller Poles and Zeros

For a PD controller, add one zero to the left of the dominant poles to attract root locus branches into the LHP. For a PI controller, add a pole at the origin (integrator) and place a zero close to it ($s = -0.1$ to -1) to eliminate steady-state error without significantly affecting transient response. For a full PID controller, combine both: add the integrator pole at the origin, one zero for the D action (fast, left of dominant poles), and one zero for the I action (slow, near origin). Adjust zero locations graphically using the Root Locus Editor to ensure all closed-loop poles remain in the LHP.

Step 4: Tune Controller Gains (Trial & Error)

Controller	Parameters	Effect on Response	Tuning Direction
PI	K_p, K_i	Eliminates steady-state error; may increase overshoot	Increase K_p for faster rise; reduce K_i to limit overshoot
PD	K_p, K_d	Faster response, reduces overshoot; does not eliminate SSE	Increase K_d to damp oscillation; increase K_p for speed
PID	K_p, K_i, K_d	Full performance: zero SSE, fast response, low overshoot	Tune K_d first, then K_p , then K_i last to minimize SSE

Step 5: Verify Closed-Loop Stability

After placing poles and zeros, verify that all closed-loop poles lie strictly in the Left Half Plane (LHP). Poles in the RHP indicate instability. Use `pole()` or check the step response for divergence. Adjust gain K using the `rlocus` gain marker until poles settle in the desired region.

Step 6: Simulate and Validate Step Response

Plot the closed-loop step response using `step(CL_sys)` and extract performance metrics with `stepinfo()`. Compare rise time, overshoot, settling time, and delay time against the design specifications. Repeat tuning if targets are not met.

Root Locus Design: Controller Configuration Summary

Type	Poles Added	Zeros Added	Achieves
PI	1 at origin (integrator)	1 near origin (e.g. $s = -0.5$)	Zero steady-state error
PD	None	1 far left (e.g. $s = -10$ to -20)	Faster response, less overshoot
PID	1 at origin (integrator)	2 zeros: one near origin, one far left	Zero SSE + fast + damped response

Results:

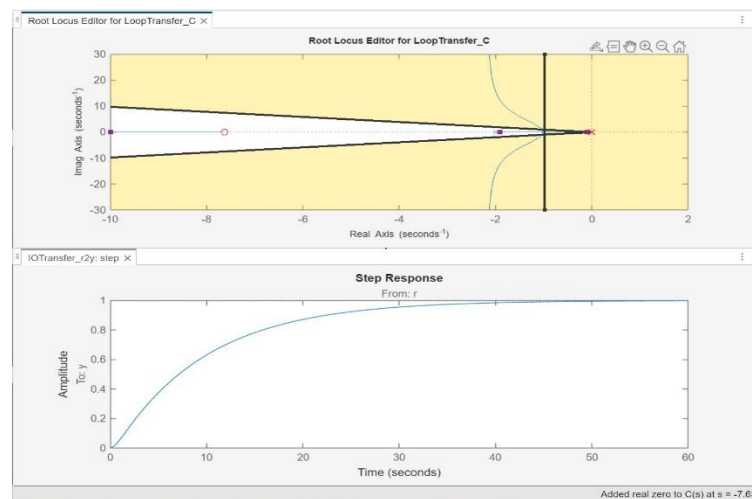


Fig 3.4: Root locus and Step response of PID controller from the editor.

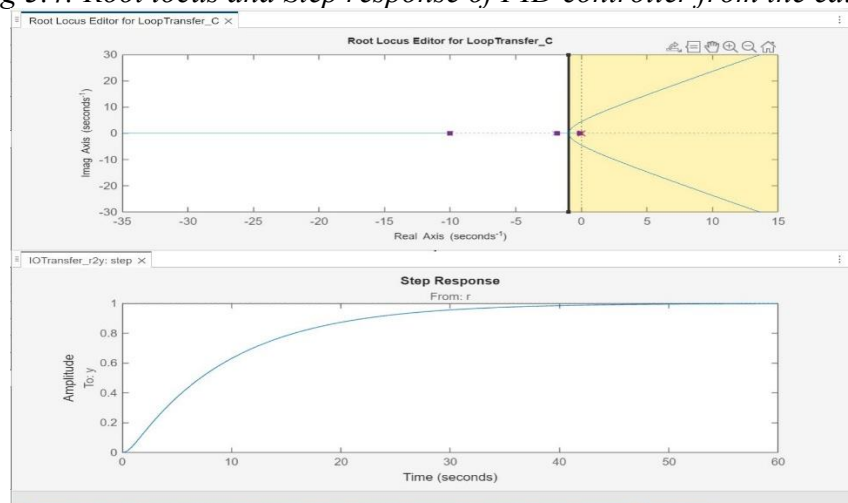


Fig 3.5: Root locus and Step response of PI controller from the editor.

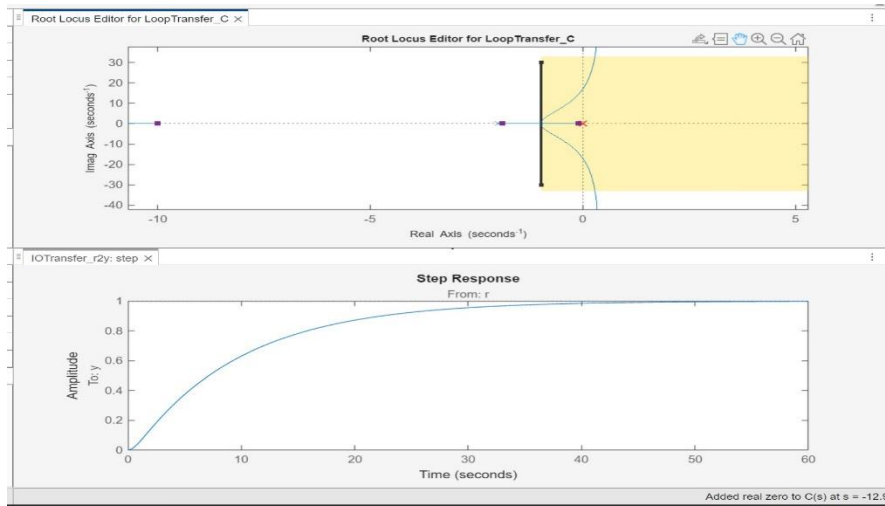


Fig 3.6: Root locus and Step response of PD controller from the editor.

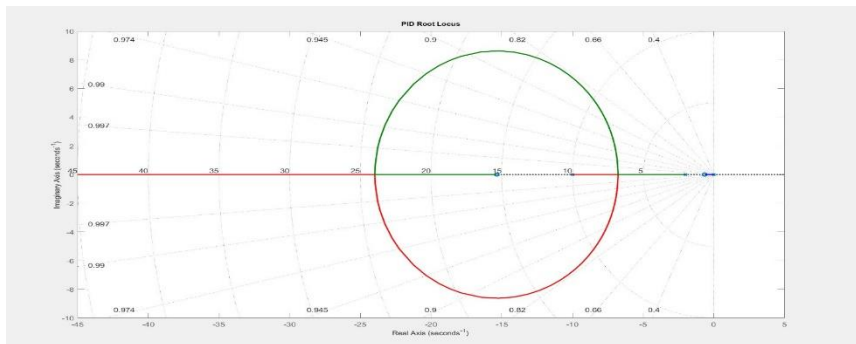


Fig 3.7: PID root locus graph from the code.

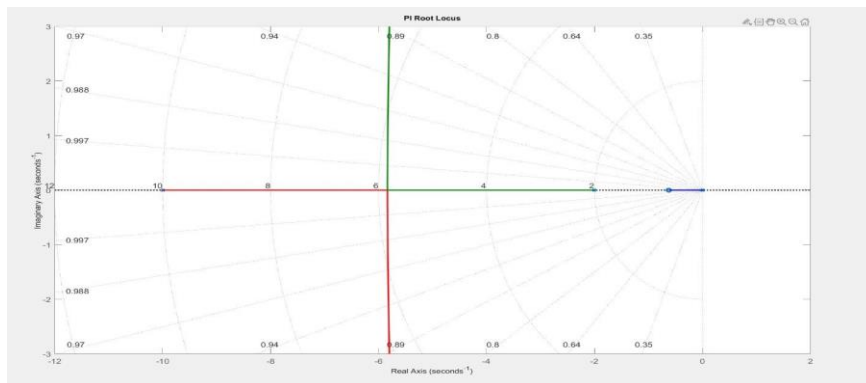


Fig 3.8: PI root locus graph from the code.

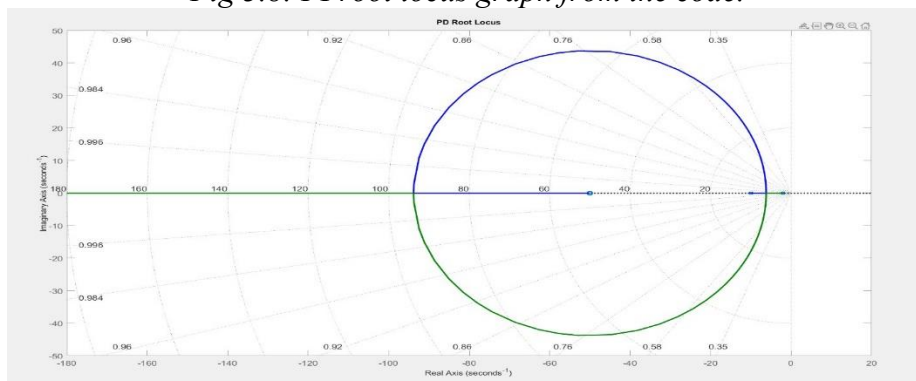


Fig 3.9: PD root locus graph from the code.

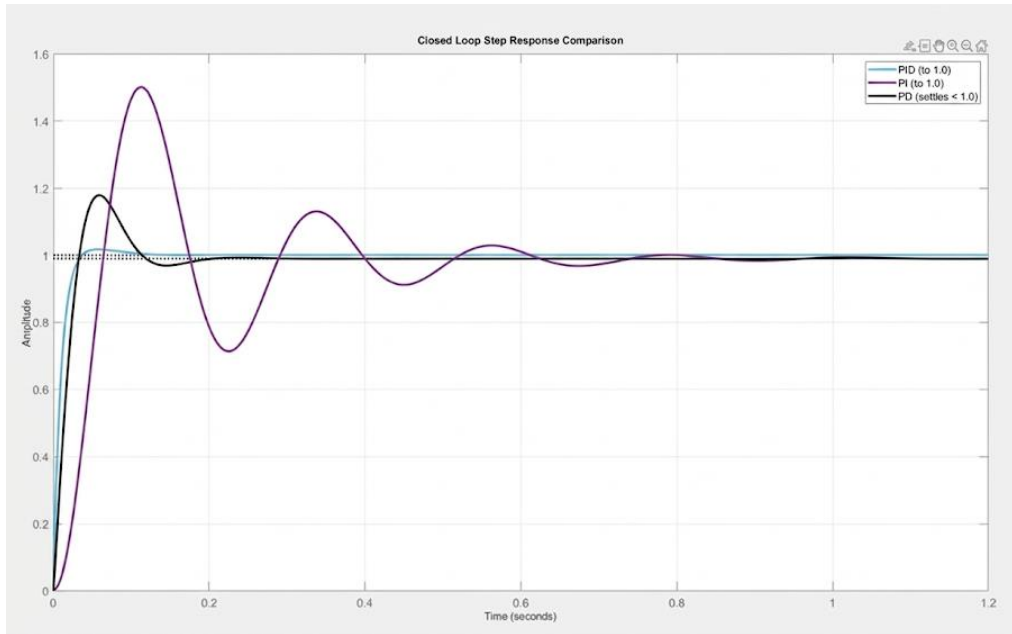


Fig 3.10: Step responses from the code.

Systems	Delay Time(s)	Rise Time(s)	Peak Time(s)	% Overshoot	Settling Time(s)
PI	0.0404	0.0429	0.1133	50.3545	0.7165
PD	0.0142	0.0250	0.0585	0	0.1497
PID	0.0073	0.0200	0.0567	1.5808	0.0311

Discussion and Conclusion:

In this experiment, the control of a DC motor was analyzed using PID, PI, and PD strategies. Initial faults were identified as system instability, characterized by a runaway response, and significant steady-state error where the output failed to reach the unit step of 1.0. These faults were attributed to the accidental placement of zeros in the Right Half Plane (RHP) and the lack of integral action in a Type 0 system. To mitigate these errors, the controller was redesigned to ensure all closed-loop poles remained in the Left Half Plane (LHP). An integrator was incorporated to eliminate steady-state error, and gains (K_p , K_i , K_d) were increased to overcome the small motor constant ($K = 0.01$). It was concluded that while PD control improved speed, only the PID and PI controllers achieved the target amplitude of 1.0. The PID controller was ultimately determined to be the superior solution, providing the best balance of stability, zero tracking error, and optimized transient performance.

References:

- [1] K. H. Ang, G. Chong, and Y. Li, "PID control system analysis, design, and technology," IEEE Trans. Control Syst. Technol., vol. 13, no. 4, pp. 559-576, 2005.
- [2] N. Salem et al., "Performance of LQR and PID Controllers for RS-550VC Motor Speed Enhancement," in Proc. IRCE, 2023.
- [3] MathWorks, "What Is Optimal Control?" [Online]. Available: <https://www.mathworks.com/discovery/optimal-control.html>